

Amlogic Android Deployment Guide

Revision: 0.1

Release Date: 2026-06-25

Copyright

© 2026 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 **Amlogic**, and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision history

Version	Change Description
v0.1 (2026-06-25)	This is the First Version.

Contents

1 Android Deployment Overview

- 1.1 NNAPI Access Scheme
- 1.2 LiteRT (AML Delegate) Access Scheme
- 1.3 JNI (NNSDK) Access Scheme
- 1.4 Android Version and Deployment Scheme Support Status
 - 1.4.1 Deployment Scheme Support
 - 1.4.2 Platform Model Support
- 1.5 Comparison of Three Deployment Schemes

2.NNAPI Deployment Scheme

- 2.1 Flowchart
- 2.2 Environment Verification
- 2.3 Quick Verification
- 2.4 NNAPI Deployment

3.LiteRT Deployment Scheme

- 3.1 Flowchart
- 3.2 Deployment Steps
 - 3.2.1 Deployment Environment Verification
 - 3.2.2 Model Preparation
 - 3.2.3 Interpreter API Deployment
 - 3.2.4 CompiledModel API Deployment

4. JNI Deployment Scheme

- 4.1 Flowchart
- 4.2 Reference Project (MobileNet Demo)
 - 4.2.1 Project Structure
 - 4.2.2 Layered Design
 - 4.2.3 JNI Encapsulation Key Points
 - 4.2.4 Kotlin API Overview
- 4.3 Deployment Steps
 - 4.3.1 Environment and Version Verification
 - 4.3.2 Prepare Native Libraries
 - 4.3.3 Compile JNI Bridge Library
 - 4.3.4 Prepare Model and Test Resources
 - 4.3.5 Inference Process
 - 4.3.6 Self-Encapsulation Guidelines
 - 4.3.7 Compilation and Execution

1 Android Deployment Overview

On the Android platform, three mature access schemes are provided for dispatching models to the Amlogic NPU for hardware acceleration: **NNAPI**, **LiteRT (AML Delegate)**, and **JNI (NNSDK)**. These three schemes have clear differences in call stack length, integration interface difficulty, supported model formats, and underlying control level. Selection should be based on a comprehensive assessment of the target device's Android system version, product form (general third-party APK/ pre-installed ROM for scheme devices/OTT customized system), performance requirements, and development efficiency.

1.1 NNAPI Access Scheme

This scheme fully reuses the LiteRT/TFLite standard development interfaces on the application side. NPU hardware acceleration can be enabled with just one line of configuration:

`setNNAPI(True)`. The complete dispatch path on the Amlogic platform is: operator requests are sent down via NNAPI, then pass through the nnservice, nnrt, and nnsdk layers sequentially, finally being dispatched to the ADLA driver for NPU hardware execution. This access method does not require integrating AML Delegate-related dependencies into the APK package, significantly reducing the application package size. However, it has clear prerequisites:

- The target device ROM must have the nnservice pre-installed and running normally.
- The nnservice, nnrt, nnsdk, and underlying driver versions must be fully compatible to avoid compatibility issues.

1.2 LiteRT (AML Delegate) Access Scheme

This scheme uses LiteRT as the core inference runtime on the application side. By mounting the dedicated **AML Delegate** component, it automatically splits and offloads all NPU-accelerable subgraphs within the `.tflite` model to the Amlogic NPU for execution. The complete dispatch path is: **AML Delegate** → **Underlying Runtime** → **NNSDK** → **ADLA Driver**, directly accessing NPU hardware resources.

The application side continues to use the previously adapted standard Interpreter or CompiledModel APIs. Enabling NPU acceleration only requires adding Delegate configuration items and declaring corresponding native dependencies.

- Compared to the NNAPI scheme, the Delegate mode has a shorter call stack, allowing for platform-level deep optimizations such as subgraph splitting, operator fusion, and inference caching strategies to be performed ahead of time during the compilation stage.
- Compared to the JNI scheme, it fully retains the LiteRT standard development paradigm, enabling seamless integration with the Google AI Edge toolchain and reducing cross-project migration costs

Applicable Scenarios: APKs or native solutions that use LiteRT as the core inference framework and require deep adaptation to Amlogic NPU capabilities.

1.3 JNI (NNSDK) Access Scheme

This scheme directly uses the Amlogic native NNSDK as the underlying inference engine. Business logic code in Java/Kotlin layers calls the underlying native library through custom JNI interfaces. NNSDK directly interfaces with ADLA to perform the entire NPU inference process, completely bypassing the LiteRT and NNAPI abstraction layers. Customers can independently encapsulate

NNSDK interfaces according to their own engineering architecture requirements, creating a customized JNI adaptation layer.

This scheme offers the shortest call stack among the three access methods, allowing for ultimate inference performance release. It is highly suitable for delivery scenarios like scheme devices or OTT customized systems that require deep customization and strong control over underlying hardware resources. However, this scheme is tightly bound to the device BSP/DDK version, and its cross-device reuse capability is limited to the same series of Amlogic chip platforms.

Applicable Scenarios: Chip solution-level delivery projects that need to completely bypass the LiteRT/NNAPI abstraction layer for extreme NPU performance tuning, or projects requiring adaptation to special custom model formats.

1.4 Android Version and Deployment Scheme Support Status

1.4.1 Deployment Scheme Support

Android Version	NNAPI	LiteRT	JNI
Android R	✓	✓	✓
Android T	✓	✓	✓
Android U	✓	✓	✓
Android 16	✓	✓	✓

1.4.2 Platform Model Support

Platform	Float32 Model	Quantized Model
A311Y3	✓	✓
Other Platforms	×	✓

A311Y3 supports both Float32 and quantized models; **Other platforms** only support quantized models (e.g., Int8 / UInt8 / int16, etc.).

1.5 Comparison of Three Deployment Schemes

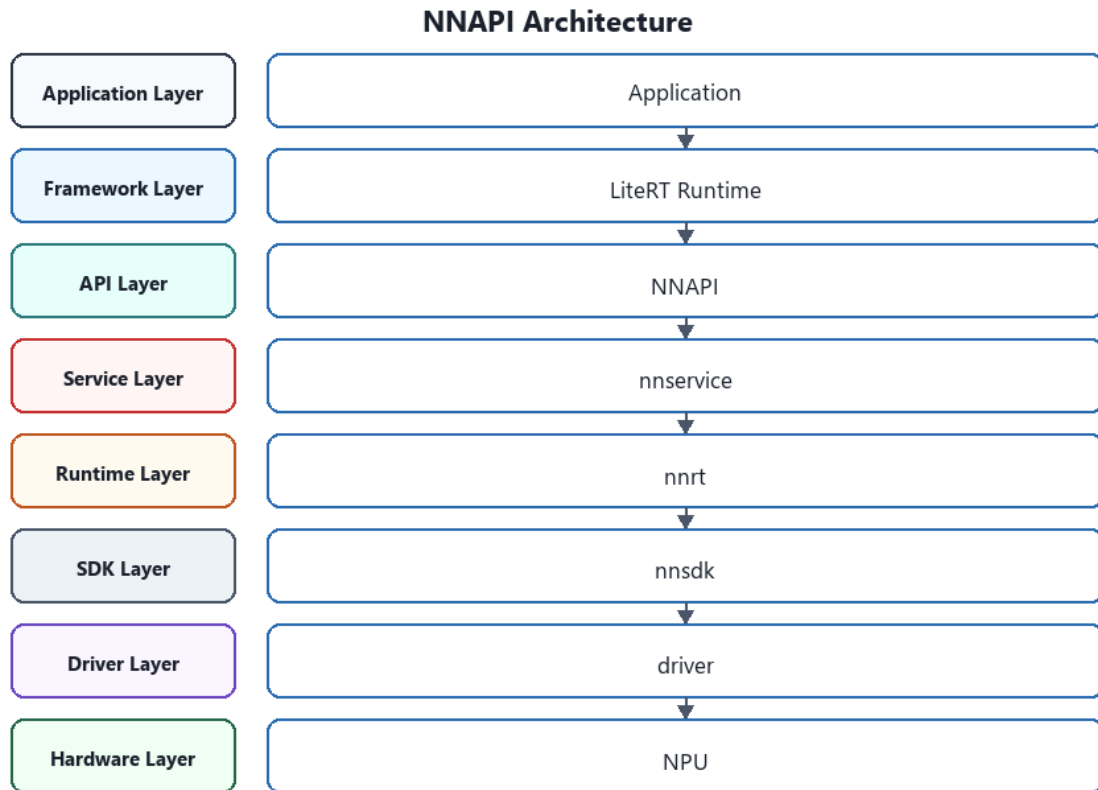
Comparison Dimension	NNAPI	LiteRT	JNI
Standard Affiliation	LiteRT NNAPI Access Paradigm + Amlogic Runtime/Service Implementation	Google LiteRT Extension Mechanism + AML Delegate Implementation	Amlogic Self-developed NNSDK, Private Stack
Model Format	Standard <code>.tflite</code> file	Standard <code>.tflite</code> file	Amlogic proprietary <code>.adla</code> format

Comparison Dimension	NNAPI	LiteRT	JNI
Operator Coverage	Limited by NNAPI common operator set and full-stack version implementation; some operators may fall back to CPU	Determined by AML Delegate, allowing for deep platform-specific optimization/fusion	Fully customizable by Amlogic NNSDK, supporting platform-specific operators
Graph Optimization & Fusion	Determined by Runtime/Service-side compilation pipeline; limited controllability on application side	Enables subgraph partitioning, fusion, quantization strategies during Delegate compilation	Determined by NNSDK compilation pipeline; controllable by solution side
Performance & Tuning	Affected by nnservice scheduling, driver policies, and multi-process sharing	Shorter stack facilitates caching, fixed-point strategies, platform-specific optimizations	Shortest stack, easy for extreme optimization, requires maintaining full stack
Compatibility & Upgrade	Strongly correlated with LiteRT version and nnservice/nnrt/nnsdk/driver version matrix	Correlated with LiteRT version + Delegate version, decoupled from NNAPI	Tightly bound to BSP/DDK/chip generation
Integration Complexity	Relatively simple on application side, relies on ROM pre-installed Runtime/Service	Requires integrating AAR/ .so, configuring Delegate parameters; many native dependencies	Requires maintaining JNI, ABI, model path, and version binding
Typical Advantages	Standard application-side API, minimal integration changes, follows TFLite/LiteRT paradigm	High performance and controllability, deep customization, facilitates platform-level optimization	Flexible customization, solution-level delivery, strong control over underlying resources
Typical Disadvantages	Relies on nnservice daemon, longer stack, issue troubleshooting requires crossing Runtime/Service/Driver	Requires maintaining version matrix for LiteRT and Delegate	Weak application-layer standardization, difficult cross-ecosystem reuse, steep learning curve

Comparison Dimension	NNAPI	LiteRT	JNI
Applicable Scenarios	Older ROM versions, solutions with pre-installed nnrt/nnservice, minimal application modification desired	Scheme devices/OTT/custom ROMs, pursuing NPU performance and controllability	Solution-level delivery, customized scenarios needing to bypass LiteRT/NNAPI

2.NNAPI Deployment Scheme

2.1 Flowchart



2.2 Environment Verification

Before deployment, verify that the device-side **nnservice**, **nnrt**, and **driver / nnsdk** are installed and version-compatible to avoid NNAPI path unavailability due to Runtime, Service, and driver mismatch.

- **nnservice Status Verification**

Verify that the system NN Service is installed and running normally, and `android.hardware.neuralnetworks-service-aidl-driver` is successfully mounted. Execute the following command

```
ps -A | grep neuralnetworks
```

- **Expected Result:** The result shows `neuralnetworks`.

- **nnrt Version Verification**

Execute the following command:

```
strings /vendor/lib64/libadla_nnrt.so | grep VERSION
```

- **Expected Result:** The returned string contains compiler version information in the format `match_nnsdk_v3_compiler_vX.Y.Z` (e.g., `match_nnsdk_v3_compiler_v3.4.0`). The extracted version number `X.Y.Z` must be $\geq$ `v3.4.0`.

- **NNSDK Version Verification**

```
strings /vendor/lib64/libnnsdk.so | grep NNSDK
```

- **Expected Results:**

- **NNSDK:** NNSDK version: v3.0.0, 2026.05, For NNSDK1 version, version number $\geq$ v3.0.0.
 - **NNSDK2:** NNSDK2 version: v1.0.0, 2026.05, For NNSDK2 version, version number $\geq$ v1.0.0.

2.3 Quick Verification

Use the **benchmark binary** provided by Google (64-bit `android_aarch64_benchmark_model`) to quickly verify if the NNAPI path is available.

下载链接: [Android benchmark model](#)

- **Run benchmark**

```
#64-bit system
./android_aarch64_benchmark_model --graph=mobilenet_v2_1.0_224_quant.tflite
--use_nnapi=true

#32-bit system
./android_arm_benchmark_model --graph=mobilenet_v2_1.0_224_quant.tflite --
use_nnapi=true
```

- **Expected Result:** The command runs successfully and exits without errors. ADLA interrupt count increases during inference.

- **NPU Interrupt Verification**

Execute the following command before and after inference:

```
cat /proc/interrupts | grep adla
```

- **Expected Result:** The interrupt number in the second column increases.

```
/data/Ci_LiteRT_Interpreter_case # cat /proc/interrupts | grep adla
91:          514          0          0          0          0          0
    0          0  GICv3 292 Level1      adla
```

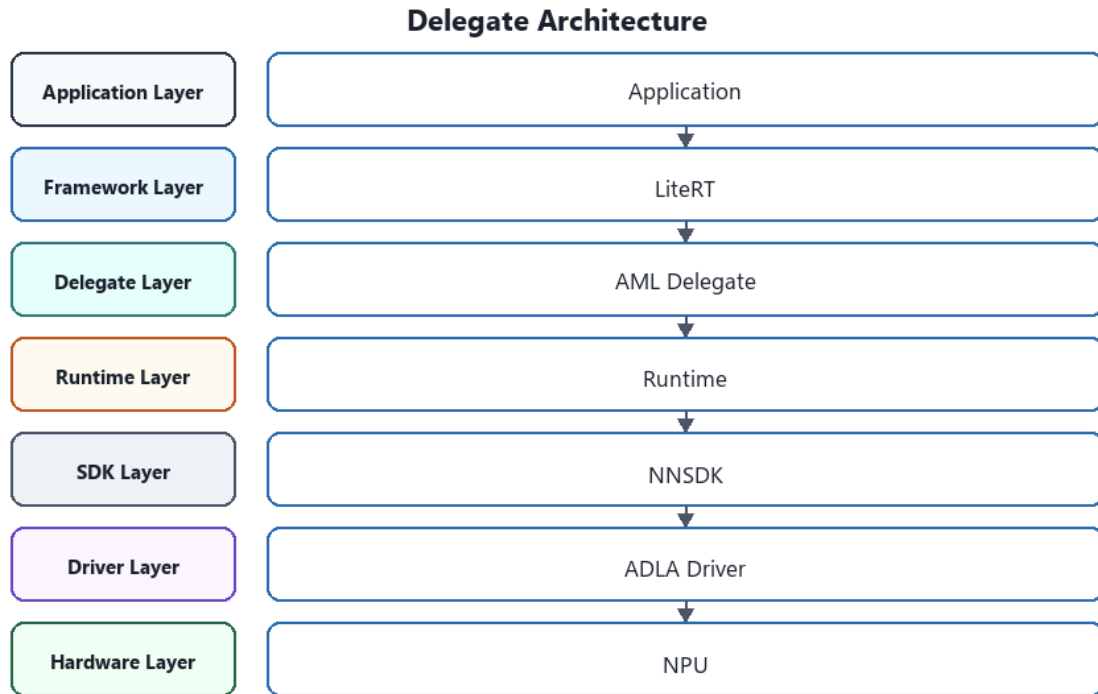
2.4 NNAPI Deployment

- **Model Preparation:** Prepare a `.tflite` model supported by the platform. For NNAPI deployment, A311Y3 supports float and int8/uint8 quantized models, while other platforms only support int8/uint8 quantized models.
- **Deployment Reference:** Refer to [image_classification](#) to configure NNAPI.

3.LiteRT Deployment Scheme

LiteRT offers two deployment schemes: Interpreter API and CompiledModel API.

3.1 Flowchart



3.2 Deployment Steps

3.2.1 Deployment Environment Verification

- **Basic Environment Verification**

```
ls /dev/ad1a0
ls /vendor/lib64/libnnsdk.so
ls /vendor/lib64/libad1a_nnrt.so
ls /vendor/lib64/libcompiler_ad1a.so
```

- **Expected Result:** The above commands execute successfully, the device node `/dev/ad1a0` and related `.so` library files all exist.

- **NNSDK Version Verification**

```
strings /vendor/lib64/libnnsdk.so | grep NNSDK
```

- **Expected Results:**
 - **NNSDK:** `NNSDK version: v3.0.0, 2026.05`, For NNSDK1 version, version number $\geq$ v3.0.0.
 - **NNSDK2:** `NNSDK2 version: v1.0.0, 2026.05`, For NNSDK2 version, version number $\geq$ v1.0.0.

- **Adla Version Verification**

```
cat /sys/class/adla/adla0/device/kmd_version
```

- **Expected Result:** ADLA Version: 2.0.2.0.0
- **AAR Package Version**
 - The downloaded aar package version number must be $\geq 3.0.0.3$. AAR package download link: [aar_package_link](#)

3.2.2 Model Preparation

- **Prepare Model:** Have the `.tflite` model artifact ready, which aligns with the operator/quantization strategy supported by AML Delegate.
- **Functional/Accuracy Regression Verification:** If needed, perform a **functional/accuracy regression** check on model outputs (CPU fallback vs. NPU output) to confirm no significant deviation introduced by quantization or operator differences.
- **Compilation/Loading Cache Note:** If the current solution uses **compilation/loading cache**, confirm the cache directory and invalidation rules in advance (to avoid issues from "old cache + new model").

3.2.3 Interpreter API Deployment

- **Quick Verification for Interpreter API Scheme**
 - **Download Link:** [Interpreter Test Bin](#)
 - **Environment Setup:** Push the existing `aml_delegate_test_bin` to an executable directory on the device (e.g., `/data/local/tmp/` or your designated workspace directory), and ensure it has execution permissions:
 - First, `adb push` the binary file to the target directory.
 - Then, execute `chmod +x <binary>` on the device.
 - **Run the Program:** `./aml_delegate_test_bin mobilenet_v2_1.0_224_quant.tflite`
 - **Expected Output:**

```
Expected output similar to the following log:
The minimum amount of memory needed for initializing the context is:
    model alloc from os          0x00000000,( 0.00 MByte)
    model alloc from drv         0x0085ae8c,( 8.36 MByte)
    model addition alloc from os 0x00031700,( 0.19 MByte)
sys mem_available 6004556 KB (5863.82 MB)
Evaluate the smmu_tlb_type is 0
[ADLAU WARN] [evaluate_addition_mem_usage:5185]
Your model file may contain unnecessary data, which will cause waste of
memory.
It is recommended to check your settings when generate the model file.
memory usage:
    model alloc from os          0x00000000,( 0.00 MByte)
    model alloc from drv         0x0085ae8c,( 8.36 MByte)
    model addition alloc from os 0x0004b2c0,( 0.29 MByte)
5.AML_Delegate AllocateTensors
6.AML_Delegate set input
6.AML_Delegate Start to do invoke
Inference status: 0
Output tensor 0
```

```

Output tensor name: Identity
Output tensor type: 3
Output tensor size (bytes): 10140
Output tensor dims: 1 2535 4
Output tensor 1
Output tensor name: Identity_1
Output tensor type: 3
Output tensor size (bytes): 202800
Output tensor dims: 1 2535 80
----- finished -----
7.CI test exit (skip interpreter teardown to avoid hang)

```

- **AAR Integration**

- Place `LiteRT_AML.aar` into the project, archive it to the module's `libs/` directory (or according to your project's convention). For detailed content about AML Delegate and LiteRT, please refer to [07 Amlogic NPU LiteRT.pdf](#).

3.2.4 CompiledModel API Deployment

- **Quick Verification for CompiledModel API Scheme:**

- Download Link: [CompiledModel Test Bin](#)
- **Environment Setup:** Push the resources to `/data/LiteRT/`, and before execution, set `LD_LIBRARY_PATH` (at least include the library directory and this directory): `export LD_LIBRARY_PATH=/vendor/lib64:/data/LiteRT:$LD_LIBRARY_PATH`
- **Run the Program:** `./aml_CompiledModel_benchmark mobilenet_v2_1.0_224_quant.tflite`
- **Expected Output:**

```

Expected output similar to the following log:
-----[AML GetInputRequirements] Enter -----
-----[AML GetOutputRequirements] Enter -----
-----[AML GetOutputRequirements] Enter -----
-----[AML GetOutputRequirements] Enter -----
***** Create CompiledModel Finished
---
Model signature count: 1
signature[0] = serving_default
-----创建输入buffer-----
-----创建输出buffer-----
-----随机填充输入数据-----
Running inference...
-----[AML RegisterTensorBuffer] Enter -----
-----[AML RegisterTensorBuffer] Enter -----
-----[AML RegisterTensorBuffer] Enter -----
-----[AML RegisterTensorBuffer] Enter -----
LiteRtDispatchInvocationContextT::AttachInputBuffer
-----[AML GetTensorBuffer] Enter -----
LiteRtDispatchInvocationContextT::AttachOutputBuffer
-----[AML GetTensorBuffer] Enter -----
LiteRtDispatchInvocationContextT::AttachOutputBuffer
-----[AML GetTensorBuffer] Enter -----
LiteRtDispatchInvocationContextT::AttachOutputBuffer
-----[AML GetTensorBuffer] Enter -----

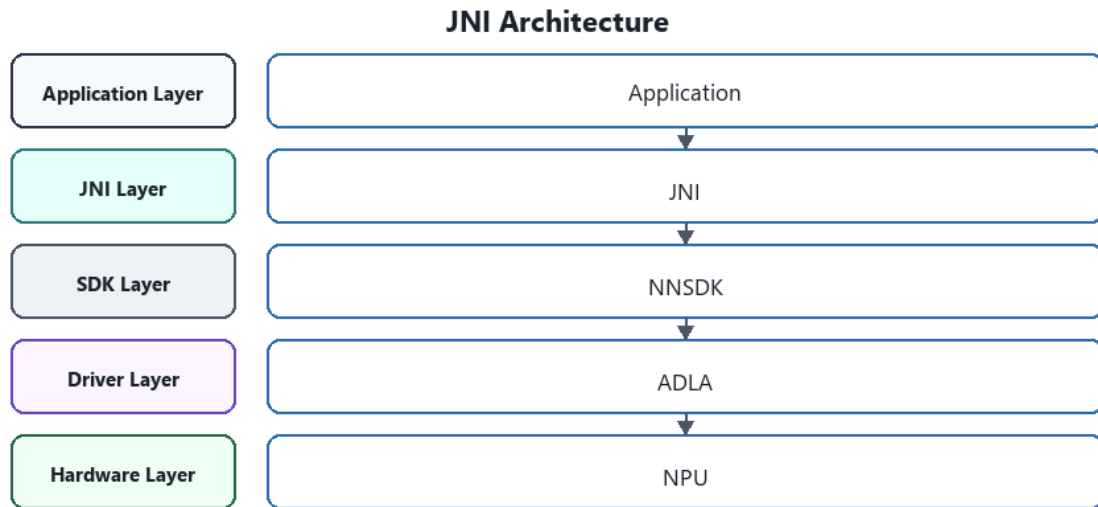
```

```
InvocationContext Execute
-----推理完成-----
output[0] size=97344
output[1] size=389376
output[2] size=24336
-----[AML GetTensorBuffer] Enter -----
-----[AML GetTensorBuffer] Enter -----
-----[AML GetTensorBuffer] Enter -----
-----[AML GetTensorBuffer] Enter -----
INFO: [accelerator_registry.cc:41] DestroyAccelerator:
ptr=0xb40000772280a6e0, name=CpuAccelerator
INFO: [accelerator_registry.cc:41] DestroyAccelerator:
ptr=0xb40000772280a680, name=NpuAccelerator
```

- **AAR Integration:** (TODO)
 - The AAR package corresponding to the CompiledModel API scheme is pending release.

4. JNI Deployment Scheme

4.1 Flowchart



4.2 Reference Project (MobileNet Demo)

A runnable JNI reference project is provided in the repository: <https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/mobilenet/jni>. It demonstrates how to integrate **NNSDK2** into an Android APK via the JNI scheme and perform image classification inference using a MobileNetV2 quantized model.

4.2.1 Project Structure

```
jni/
├── app/
│   ├── CMakeLists.txt           # Compiles libnnsdk_jni.so
│   ├── build.gradle.kts         # ABI, jniLibs, CMake configuration
│   └── src/main/
│       ├── assets/demo/         # Model, labels, test images
│       │   ├── mobilenet_v2_1.0_224_quant.adla
│       │   ├── labels.txt
│       │   └── *.jpg / *.jpeg
│       ├── java/com/example/NN_JNI/
│       │   ├── NnSdk2.kt         # NNSDK2 Kotlin encapsulation
│       │   ├── NnStruct.kt       # Struct read/write helpers
│       │   ├── ImagePreprocessor.kt # Image preprocessing & quantization
│       │   ├── MobilenetDemo.kt  # Inference process encapsulation
│       │   ├── DemoAssetHelper.kt # Assets resource copying
│       │   └── MainActivity.kt    # Compose UI
│       └── jni/
│           ├── nnsdk_jni.cpp     # JNI implementation (dlopen libnnsdk.so)
│           └── nnsdk2.h          # NNSDK2 C API header file
│       └── libs/arm64-v8a/       # Native libraries provided by the
solution side
│   ├── libadla.so
│   └── libnnsdk.so
```

4.2.2 Layered Design

Layer	Module	Responsibilities
UI Layer	MainActivity	Image selection, preview, trigger inference, display Top-5 results
Business Layer	MobilenetDemo	Orchestrates the full process: init → preprocess → infer → postprocess
Encapsulation Layer	NnSdk2 / NnStruct	Exposes Kotlin API to the outside
JNI Layer	nnsdk_jni.cpp	dlopen loads libnnsdk.so, RegisterNatives registers native methods
SDK Layer	libnnsdk.so	NNSDK2 inference interfaces (amlnn_init / run / outputs_get, etc.)
Driver Layer	libadla.so	ADLA NPU driver

Call Stack:

```
MainActivity → MobilenetDemo → NnSdk2 → nnsdk_jni → libnnsdk.so → libadla.so → NPU
```

4.2.3 JNI Encapsulation Key Points

The reference project uses a **dynamic loading** approach to access NNSDK. The JNI layer does not statically link `libnnsdk.so`; instead, symbols are resolved via `dlopen` / `dlsym` at runtime.

Advantages:

- The JNI project only compiles `libnnsdk_jni.so`, keeping it very small.
- `libnnsdk.so` / `libadla.so` are provided by the solution side, facilitating independent upgrades from BSP versions.
- Symbol registration and error handling can be centralized in `JNI_OnLoad`.

Library Loading Order (`NnSdk2.kt`):

1. `system.loadLibrary("adla")` — Preloads ADLA driver.
2. `system.loadLibrary("nnsdk_jni")` — Loads JNI bridge library.

JNI Registration Method: In `JNI_OnLoad`, use `RegisterNatives` to bind C++ functions to the `native*` methods of `NnSdk2`, avoiding the need for `Java_com_example_...` naming conventions.

4.2.4 Kotlin API Overview

The inference interfaces exposed by `NnSdk2` are as follows:

Method	Description
--------	-------------

Method	Description
<code>initFromFile(modelPath, ...)</code>	Loads <code>.ad1a</code> model from file path, returns context handle
<code>initFromBuffer(modelBuffer, ...)</code>	Loads model from memory buffer
<code>query(context, queryCmd, info)</code>	Queries SDK version, input/output attributes, etc.
<code>inputsSet(context, buffers, sizes, indices)</code>	Sets input tensors (Direct <code>ByteBuffer</code>)
<code>run(context)</code>	Executes inference
<code>outputsGet(context, index)</code>	Gets output, returns <code>FloatArray</code> (SDK internally dequantizes)
<code>destroy(context)</code>	Releases context

Note: `outputsGet` does not take a pre-allocated buffer. Instead, NNSDK internally populates `buf / size` within `amlnn_outputs_get`, and the JNI layer copies it into a Java `FloatArray`. This is the correct usage consistent with some C++ Demos. Pre-allocating your own buffer may result in all-zero outputs.

4.3 Deployment Steps

4.3.1 Environment and Version Verification

Before deployment, verify that the device-side ADLA driver and NNSDK are ready, using the same methods as in [2.2 Environment Verification](#):

```
# NNSDK version
strings /vendor/lib64/libnnsdk.so | grep NNSDK

# ADLA interrupts (should increase during inference)
cat /proc/interrupts | grep ad1a
```

Item	Requirement
Android Version	ndroid R and above
ABI	<code>arm64-v8a</code> (reference project only packages this architecture)
NNSDK	NNSDK2 v1.0.0 and above
Model Format	<code>.ad1a</code> (independently compiled on-device artifact)

4.3.2 Prepare Native Libraries

Place the so files provided by the solution side into the project's jniLibs directory:

```
app/src/main/libs/arm64-v8a/  
├─ libadla.so  
├─ libnnsdk.so  
└─ ...
```

In `build.gradle.kts`, specify the jniLibs path via `sourceSets`:

```
sourceSets {  
    getByName("main") {  
        jniLibs.srcDirs("src/main/libs")  
    }  
}  
  
ndk {  
    abiFilters += "arm64-v8a"  
}
```

4.3.3 Compile JNI Bridge Library

`app/CMakeLists.txt` compiles `libnnsdk_jni.so`:

```
add_library(nnsdk_jni SHARED src/main/jni/nnsdk_jni.cpp)  
target_link_libraries(nnsdk_jni PRIVATE ${log-lib} dl)
```

`build.gradle.kts` enables external CMake build:

```
externalNativeBuild {  
    cmake {  
        path = file("CMakeLists.txt")  
    }  
}
```

The compiled artifact `libnnsdk_jni.so` is installed with the APK; at runtime, it dynamically links to NNSDK via `dlopen("libnnsdk.so")`.

4.3.4 Prepare Model and Test Resources

Place the model, labels, and test images into `assets/demo/`:

```
assets/demo/  
├─ mobilenet_v2_1.0_224_quant.adla  
├─ labels.txt  
├─ cat_224x224.jpg  
├─ dog_224x224.jpg  
└─ fish_224x224.jpeg
```

When the application starts, assets are automatically copied to the application's private directory (`getExternalFilesDir(null)/demo/`). The model is loaded using an absolute path during inference:

```
val modelPath = DemoAssetHelper.getModelFile(context).absolutePath
val contextHandle = NnSdk2.initFromFile(modelPath, backendType =
    NnSdk2.BACKEND_ADLA_NPU)
```

4.3.5 Inference Process

Referring to `MobilenetDemo.kt`, the complete inference process consists of the following steps:

1. init	amlnn_init + query input/output tensor attributes
2. preprocess	Load image → resize 224×224 → RGB quantization
3. input_set	inputsSet(Direct ByteBuffer)
4. run	amlnn_run
5. output_get	outputsGet → FloatArray logits
6. postprocess	softmax → Top-K classification
7. release	destroy(context)

Example code:

```
// 1. Initialize
val handle = NnSdk2.initFromFile(modelPath, backendType =
    NnSdk2.BACKEND_ADLA_NPU)

// 2. Query input attributes
val attrBuffer = NnStruct.createTensorAttrBuffer(0)
NnSdk2.query(handle, NnSdk2.QUERY_INPUT_ATTR, attrBuffer)
val inAttr = NnStruct.readTensorAttr(attrBuffer)

// 3. Preprocess
val inputBuffer = ImagePreprocessor.preprocessQuant(bitmap, inAttr.scale,
    inAttr.zp, inAttr.type)

// 4. Inference
NnSdk2.inputsSet(handle, arrayOf(inputBuffer), intArrayOf(inAttr.nElems))
NnSdk2.run(handle)
val logits = NnSdk2.outputsGet(handle, 0)

// 5. Release
NnSdk2.destroy(handle)
```

4.3.6 Self-Encapsulation Guidelines

Customers can adapt or extend the JNI layer based on the reference project. It is recommended to follow these principles:

Principle	Description
Dynamically Load NNSDK	Use <code>dlopen</code> / <code>dlsym</code> to avoid version coupling from static linking.
Direct ByteBuffer for Input	<code>inputsSet</code> requires Direct Buffer; regular heap buffer will be invalid.
Output Populated by SDK	<code>outputsGet</code> does not set <code>buf</code> ; <code>am1nn_outputs_get</code> internally allocates and returns it.
Struct Alignment	Buffers used by <code>query</code> must get size via <code>nativesizeof</code> and be read/written in native byte order.
Thread Safety	A single context is not recommended for multi-threaded concurrent <code>run</code> ; use separate contexts for multiple models.
Version Binding	<code>nnsdk2.h</code> , <code>.ad1a</code> model, and <code>libnnsdk.so</code> must match the BSP compiler version.

4.3.7 Compilation and Execution

```
# Compile APK
cd 01_MobileNet
./gradlew assembleDebug

# Install on device
adb install -r app/build/outputs/apk/debug/app-debug.apk

# View inference logs
adb logcat -s MobilenetDemo Nnsdk2 Nnsdk2Jni
```

After installation, open the **MobileNet Demo** app: select a test image → click "Start Detection" → view the Top-5 classification results. Detailed logs can be obtained via `adb logcat`.